Red Social Pascualina

Informe Técnico Comparativo Multi-SGBD y NoSQL

Tarea 4 - Integración de PostgreSQL, MySQL, SQL Server, Big Data e IoT

Integrantes del Grupo 6:

Eisson Javier Oyola Tique
Julian Quintero Copete
Jonathan Andres Serna Velasco
Daniel Alejandro Stuart Muñoz

Institución Universitaria Pascual Bravo
Facultad de Ingeniería - Departamento de Sistemas
Medellín, Colombia

13 de octubre de 2026

Índice

1. Introducción y Objetivos **2**

2. Justificación de la Elección de los Tres SGBD **2**

3. Integración NoSQL para Big Data e IoT **2**
3.1. Caso de Uso 1: Analítica de Interacciones (Big Data) 2
3.2. Caso de Uso 2: Telemetría de Sensores en el Campus (IoT) 3

4. Análisis Comparativo y Diferencias de Sintaxis DDL **3**
4.1. Dificultades Sintácticas y Soluciones Aplicadas 3

5. Conclusiones Individuales y Reflexión Crítica **3**
5.1. Eisson Javier Oyola Tique 3
5.2. Julian Quintero Copete 4
5.3. Jonathan Andres Serna Velasco 4
5.4. Daniel Alejandro Stuart Muñoz 4
5.5. Conclusión Grupal y Oportunidades de Mejora 4

1. Introducción y Objetivos

El presente informe técnico representa la fase final del desarrollo de la infraestructura de datos para la **Red Social Estudiantil Pascualina**. Tras haber cubierto el modelado conceptual, relacional (3FN) e implementación física en PostgreSQL, esta etapa expande las capacidades del sistema hacia un entorno **Multi-SGBD** (PostgreSQL, MySQL y Microsoft SQL Server) e integra capacidades de datos semiestructurados (**NoSQL / JSON**) para responder a escenarios modernos de analítica masiva de datos (Big Data) e Internet de las Cosas (IoT).

Los objetivos principales de este informe son:

Evaluar el proceso de portabilidad del lenguaje DDL entre tres motores relacionales líderes en el mercado.

Justificar e implementar campos semiestructurados de tipo `JSON` / `JSONB` para analítica de interacciones y sensores del campus.

Realizar un análisis comparativo de sintaxis, rendimiento, flexibilidad y dificultades encontradas durante la migración Multi-SGBD.

2. Justificación de la Elección de los Tres SGBD

La selección de los motores responde a la necesidad de contrastar distintos paradigmas de licenciamiento, arquitecturas de almacenamiento y filosofías de diseño:

1. **PostgreSQL (SGBD 1 - Open Source Enterprise):** Destacado por su estricto cumplimiento del estándar ANSI SQL y su manejo nativo avanzado de objetos binarios `JSONB` con soporte para índices GIN.
2. **MySQL (SGBD 2 - Web & Cloud Popularity):** Ampliamente utilizado en aplicaciones web por su motor transaccional `InnoDB`, ofreciendo ligereza y soporte de tipo `JSON` nativo a partir de la versión 5.7+.
3. **Microsoft SQL Server (SGBD 3 - Entorno Corporativo T-SQL):** Representante del ecosistema comercial propietario con Transact-SQL. Implementa validador `ISJSON()` sobre columnas de texto extendido `NVARCHAR(MAX)`.

3. Integración NoSQL para Big Data e IoT

Para evitar la rigidez del esquema relacional estricto frente a volúmenes masivos de datos no estructurados o de alta velocidad, se integraron dos atributos clave:

3.1. Caso de Uso 1: Analítica de Interacciones (Big Data)

Tabla afectada: `Publicaciones`

Campo implementado: `metadatos_interaccion` (`JSONB` / `JSON` / `NVARCHAR(MAX)`)

Justificación técnica: Permite almacenar registros de actividad en formato Key-Value dinámico (dispositivo, sistema operativo, tiempo de retención en pantalla, coordenadas aproximadas de lectura, clics en enlaces y tags dinámicos) sin necesidad de ejecutar alterar esquemas relacionales al añadir nuevos eventos de analítica.

3.2. Caso de Uso 2: Telemetría de Sensores en el Campus (IoT)

Tabla afectada: `Eventos`

Campo implementado: `telemetria_aforo` (`JSONB` / `JSON` / `NVARCHAR(MAX)`)

Justificación técnica: Permite la ingesta continua de lecturas desde sensores físicos instalados en aulas y auditorios universitarios (conteo de aforo en tiempo real, temperatura ambiente, nivel de humedad y decibelios de ruido) durante la ejecución de tutorías o hackatones.

4. Análisis Comparativo y Diferencias de Sintaxis DDL

A continuación se resumen las diferencias técnicas y sintácticas halladas durante la traducción del script DDL a los tres motores:

Tabla 1: Cuadro comparativo de sintaxis y características físicamente implementadas.

Característica	PostgreSQL	MySQL (InnoDB)	SQL Server (T-SQL)
Autoincremental	`SERIAL` / `BIGSERIAL`	`AUTO_INCREMENT`	`IDENTITY(1,1)`
Marcas de Tiempo	`TIMESTAMP`	`DATETIME`	`DATETIME2`
Manejo de Cadenas	`VARCHAR` / `TEXT`	`VARCHAR` / `TEXT`	`NVARCHAR` (Unicode)
Tipo de Dato NoSQL	`JSONB` (Binario optimizado)	`JSON` (Nativo binario)	`NVARCHAR(MAX)` + `ISJSON()`
Limpieza de Tablas	`DROP TABLE ... CASCADE`	`SET FOREIGN_KEY_CHECKS=0;`	`IF OBJECT_ID(...) IS NOT NULL`

4.1. Dificultades Sintácticas y Soluciones Aplicadas

1. **Restricciones de Claves Foráneas en SQL Server:** SQL Server bloquea la creación de múltiples rutas de borrado en cascada (*Multiple Cascade Paths*). Para solucionarlo en la tabla `Conexiones` y `Comentarios`, se cambió la regla a `ON DELETE NO ACTION`.

2. **Validación de Documentos JSON:** Mientras PostgreSQL y MySQL validan nativamente la sintaxis JSON en el momento del almacenamiento, SQL Server requirió la inclusión de restricciones explícitas del tipo `CHECK (ISJSON(campo) >0)`.

5. Conclusiones Individuales y Reflexión Crítica

5.1. Eisson Javier Oyola Tique

La transición a un entorno Multi-SGBD demostró que, aunque el estándar ANSI SQL sienta las bases, cada motor posee particularidades críticas en su arquitectura. Trabajar con el tipo `JSONB` en PostgreSQL me permitió entender el valor de los modelos híbridos (Relacional + NoSQL) para optimizar consultas de analítica avanzada sin perder la integridad ACID.

5.2. Julian Quintero Copete

El reto más interesante en esta fase fue adaptar las reglas de borrado en cascada para SQL Server. Comprender las restricciones de dependencias circulares en T-SQL reforzó mi criterio para diseñar modelos físicos verdaderamente portables y tolerantes a las limitaciones de motores comerciales.

5.3. Jonathan Andres Serna Velasco

Evaluar el comportamiento de MySQL con su motor InnoDB frente a PostgreSQL me ayudó a identificar cómo influyen los índices y la estructuración de tipos de datos en entornos cloud. La integración de los metadatos de interacciones demostró cómo una tabla relacional puede adaptarse a escenarios de Big Data sin romper la normalización.

5.4. Daniel Alejandro Stuart Muñoz

El diseño de la captura de telemetría IoT para eventos del campus demostró la practicidad de almacenar objetos JSON flexibles dentro de bases de datos relacionales. Esta experiencia fortalece de manera significativa nuestras competencias profesionales para diseñar arquitecturas de datos modernas, escalables e interoperables.

5.5. Conclusión Grupal y Oportunidades de Mejora

Como grupo, concluimos que la combinación de un núcleo relacional bien normalizado (3FN) con columnas semiestructuradas (JSON/JSONB) ofrece el equilibrio perfecto entre consistencia y rendimiento. Como mejora futura, se propone la creación de índices GIN en PostgreSQL e índices sobre columnas calculadas en SQL Server/MySQL para maximizar la velocidad de búsqueda en los atributos NoSQL.